IN C++

Conditional Statements & Loops

DEFINITION

A conditional statement is a program construct that lets you make a decision about which part of the code should run.

The Five Types

1. if-else
2. if-else (nested)
3. if-else if ... else
4. switch
5. ternary

1. if-else

C++

```
int age = 31;
if (age >= 18) {
    cout << "He can vote";
}
else {
    cout << "He cannot";
}
```

★NOTE

- **else cannot exist without an if.**
- **if can exist on its own, without an else.**

C++

```
int age = 30;
if (age < 50) {
    cout << "You are young";
}
```

Comparison Operators

ROUGH WORK

Inside an if( ) parenthesis there should be a condition — a condition which results in true or false.

```cpp
10 > 3          // True
3 == 4          // False
"ah" == "hi"    // False
```

Comparison Operators

Operator	Meaning
>=	More than or equal
<=	Less than or equal
==	Equal to
!=	Not equal to
<	Less than
>	Greater than

3. if-else if ladder

```cpp
if (C1) {
    // some code
}
else if (C2) {
    // code
}
else if (C3) {
    // code
}
else {
    // code
}
```

★NOTE

- **An if-else if ladder can be complete without a final else.**
- **Best practice: add a default/final else to handle edge cases, or random/wrong inputs, in real-world scenarios.**

Nested if

```cpp
int age = 20;
```

```
bool has_licence = true;

if (age >= 18) {
    if (has_licence) {
        cout << "can drive";
    }
    else {
        cout << "cannot drive";
    }
}
else {
    cout << "Too young";
}
```

How the ladder actually evaluates

> ★NOTE
>
> **If an if fails, the else if's below it are checked next, in sequential order — and the final else runs only if every condition above it has failed.**
>
> - **Every if in a plain, independent block checks its own condition regardless of what came before.**
> - **An else if only gets checked when the condition above it in the same ladder has failed.**

Compare two versions of the same idea — separate, independent if statements versus a proper if-else if ladder:

C++

```
// Version A: independent ifs (NOT a ladder)
int a = 10;
if (a > 3) { cout << "A"; }
if (a > 2) { cout << "B"; }
if (a > 2) { cout << "E"; }
// every condition is checked, so all three can fire
```

Output: ABE

C++

```
// Version B: if-else if ladder
int a = 10;
if (a > 3)      { cout << "A"; }
else if (a > 2) { cout << "B"; }
else            { cout << "E"; }
// first true branch wins, the rest are skipped
```

Output: A

Logical Operators

Logical Operators

Symbol	Meaning
&&	AND
||	OR
!	NOT

&& (AND) — true only when both sides are true:

AND Truth Table

A	B	A && B
1	1	True
1	0	False
0	1	False
0	0	False

|| (OR) — true when at least one side is true:

OR Truth Table

A	B	A || B
0	0	False
0	1	True
1	0	True
1	1	True

4. switch

C++

```
switch (expr) {
    case v1:
        statement1;
        break;
    case v2:
        statement2;
        break;
    default:
        statements;
}
```

★NOTE

- **Why use break? Without it, every case below the one that matches also runs.**
- **default catches whatever value doesn't match any listed case.**
- **expr can be an integer or an enum — not just any type.**

Operator Precedence (High → Low)

Operator Precedence

Level	Operators
1 (highest)	()
2	* / %
3	+ -
4	< <= > >=
5	== !=
6	&&
7	||
8	?: (ternary)
9 (lowest)	=

5. Ternary

```cpp
condition ? expIfTrue : expIfFalse;
```

A compact, inline stand-in for a simple if-else that returns a value.

Loops

DEFINITION

A loop repeats a block of code until a stated condition becomes false, so you don't have to write the same statements out by hand.

1. for loop

A for loop packs three parts into one line, separated by semicolons: an initialization (runs once, before anything else), a condition (checked before every pass), and an update (runs after every pass).

```cpp
for (int i = 0; i < 5; i++) {
    cout << i << " ";
}
```

Output: `0 1 2 3 4`

Step by step: i starts at 0. Before each pass, i < 5 is checked — if true, the body runs and prints i. After the body, i++ runs, then the condition is checked again. Once i reaches 5, the condition is false and the loop stops.

★NOTE

- **All three parts (init; condition; update) are optional, but the two semicolons always stay — for(;;) is valid C++.**
- **Leaving the condition out (or making it always true) with no break inside creates an infinite loop.**

2. while loop

A while loop only has a condition — you set up and update the counter yourself, outside the loop line.

```cpp
int i = 0;
while (i < 5) {
    cout << i << " ";
    i++;
}
```

Output: `0 1 2 3 4`

The condition is checked before every single pass, including the very first one. If i < 5 were false to begin with, the body would not run even once.

3. do-while loop

```cpp
int i = 0;
do {
    cout << i << " ";
    i++;
} while (i < 5);
```

Output: `0 1 2 3 4`

★NOTE

do-while always runs its body at least once, because the condition is checked after the first pass, not before it.

- **while and for check the condition first, so they can run zero times.**
- **do-while checks last, so the body always executes at least once — useful for things like "ask the user for input, then re-ask if it was invalid."**

4. break and continue

break exits the loop immediately, skipping every remaining iteration:

```
C++
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    cout << i << " ";
}
```

Output: `0 1 2 3 4`

Once i becomes 5, break fires and the loop ends right there — 5, 6, 7, 8, 9 are never printed.

continue skips only the current pass and jumps straight to the update step, then re-checks the condition:

```
C++
for (int i = 0; i < 5; i++) {
    if (i == 2) continue;
    cout << i << " ";
}
```

Output: `0 1 3 4`

When i is 2, continue skips the cout for that pass only — the loop itself keeps running, so 3 and 4 still print.

5. Nested loops

```
C++
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i << j << " ";
    }
}
```

Output: `11 12 13 21 22 23`

The inner loop runs all the way through (j = 1, 2, 3) for every single pass of the outer loop, before the outer loop moves on to its next value of i.

6. Range-Based for loop (C++11)

```C++
vector<int> nums = {10, 20, 30};
for (int n : nums) {
    cout << n << " ";
}
```

Output: `10 20 30`

Reads naturally as "for each n in nums." There's no index variable to manage and no risk of an off-by-one error — n takes each element's value in turn, in order.

for vs while vs do-while

Loop	Condition checked	Minimum runs
for	Before each pass	0
while	Before each pass	0
do-while	After each pass	1

★ NOTE ●

End of Notes — Conditional Statements & Loops

NextGen Data Minds